

SRD Logistics Service Performance & Live Reporting Dashboard

Solution architecture, customer requirements and frequently asked questions

Purpose

SRD Logistics asked for one system that measures service percentage, target achievement, route variation, journey-stage delays, customer performance and live operational movement, with drill-down and actionable improvement suggestions. This document explains how we will build that as a web dashboard, what is already working, and what we need from SRD Logistics to move from demonstration to production use.

Where we are today

A working dashboard covering every area of SRD's requirement document has been built and tested on **synthetic data** that mimics SRD's LR (lorry receipt) structure. It runs in a web browser and includes an optional machine-learning delay-risk score. The next step is to connect it to SRD's real data and live APIs, which is what the Upload page, database and API connection in this document provide.

SRD requirement area	Dashboard page	What users get
1. Service % and target performance	Service vs Target	Source, destination, route, via, business, consignor and consignee service % against configurable targets; target-day view (4, 5, 6, 7 and above)
2. Source / destination / via / route performance	Network Performance	Performance and delivery reports, highest-to-lowest delay rankings, business-wise achievement
3. Delay analysis and drill-down	Delay Analysis	Delay reason and % at each journey point; route to stage to LR to reason, with IF-condition and operational remarks
4. Specific LR / customer analysis	LR & Customer	LR search with full journey, consignor and consignee reports and variation from target
5. Period-wise and comparison reports	Period Reports	Weekly, 10-day, 15-day, monthly, yearly; any two periods compared
6. Booking to main hub dispatch	Booking to Main Hub	Pending, delayed and dispatched LRs with ageing; source and hub performance
7. Live report on SRD software API	Live Operations	Daily bookings, lagging / leading, target achievement, weight, godown stock, hub movement
8. System-generated suggestions	Improvement Suggestions	Route-wise actions from recurring delays, drilling down to stage and LRs
9. Reporting and dashboard rules	Filters and Configuration	Period, source, destination, via, business, consignor, consignee, LR and target-day filters; editable targets and rules

Delivered beyond the requirement document: the ML delay-risk page (which open LRs are likely to miss target, and why).

End-to-end architecture

Data flows top to bottom: SRD data arrives, is validated and stored, is analysed, and is shown to users in a browser.



BUILT Working today in the demonstration

PARTLY BUILT Code exists; needs SRD's real API details

TO BUILD Part of the proposed build once data arrives

CUSTOMER Provided by SRD Logistics

OPTIONAL Only if SRD already has this

DECISION To be agreed with SRD

Logos are trademarks of their respective owners and are shown only to identify the technologies used.

How the layers fit together

Layer	Technology	What it does	Status
Data sources	Excel / CSV, SRD application APIs	SRD supplies LR extracts, master data and target definitions, and API access for live data.	SRD provides
Ingestion	Streamlit page, Python, pandas	Checks each file against the LR template (required columns, date order, duplicate LR numbers, unknown routes), shows a validation report, and loads only after the user confirms. The live connector polls SRD's APIs on a schedule.	To build (connector partly built)
Storage	PostgreSQL	Permanent home for LR records, route targets, rules and an upload log. Re-uploading a file updates existing LRs instead of duplicating them.	To build
Processing	Python, pandas, NumPy, scikit-learn	Calculates service %, variation, delay point and reason, period and hub measures, improvement suggestions and the delay-risk score. Results always reflect the stored targets.	Built
Presentation	Streamlit, Plotly	11 interactive pages with global filters, drill-down and CSV export, opened by a web link.	Built
Hosting	GitHub; Streamlit Community Cloud or private cloud / on-premise	Code is versioned in GitHub. Community Cloud suits a demo or pilot; production data should sit on SRD-approved private hosting.	To be agreed

Hosting options

	Streamlit Community Cloud	Private cloud or on-premise
Best for	Demonstration and pilot with sample or anonymised data	Production use with real LR and customer data
Cost	Free tier	Server or cloud subscription (SRD or DataQ managed)
Access control	Public link by default; private sharing needs approved email invites	Company login (SSO) and network restrictions available
Data safety	Temporary disk; data must live in an external database	Data stays inside SRD-approved infrastructure
Our recommendation	Use for the pilot	Use for go-live

Key design principles

- **Targets and rules are configurable.** Benchmark days and target % per route, hub time limits and suggestion thresholds are edited on screen and never hard-coded.
- **One source of truth.** Every page reads the same stored data, so numbers agree across reports.
- **Drill-down everywhere.** Summary to route, source, destination, via, customer and individual LR.
- **Safe by default.** The live connector falls back to the last good data if SRD's API is unavailable, and uploads are validated before anything is saved.

Your questions, answered

Answers below describe the proposed production design. Items marked **(to build)** are included in the build once SRD shares its data.

1. How will data be fetched from Excel? Will there be a user interface for the Excel loader?

Yes. A dedicated **Upload & Validate** page (to build) lets an authorised user:

- choose an Excel or CSV file (we provide a template that matches SRD's own export headings);
- map SRD's column names to the dashboard's fields once, and save that mapping for next time;
- review a validation report: missing columns, duplicate LR numbers, delivery before booking, unknown routes or hubs;
- preview the rows, then click **Load**. Nothing is saved until the user confirms.

For automatic loading, the same database can be fed by SRD's APIs (see page 2) or by an ETL job if SRD already has one.

2. Once data is ingested, where is it stored?

In a **PostgreSQL database** (to build), not in the application's files. The main tables are LR records, route targets, business rules, an upload log (who loaded what, when, how many rows) and a model registry. The database can be hosted by SRD, by DataQ, or on a managed cloud service, with daily backups. This matters because Streamlit Community Cloud's disk is temporary: anything saved only in the app's files is lost on restart.

3. How is the data processed and kept persistent?

- **Load:** validated rows are inserted or updated by LR number, so re-uploading a corrected file fixes records instead of duplicating them.
- **Targets and rules:** saved in the database, so edits made on the Configuration page survive restarts and apply to every report.
- **Calculation:** service %, variation, delay point and reason, period and hub measures are calculated by the Python analytics engine from the stored LRs and refreshed after each load. Changing a target changes the results without reloading any data.
- **Machine learning:** the delay-risk model is retrained after loads and stored with its training date and accuracy.
- **Audit:** the upload log keeps a history of every load.

4. How does the presentation layer access the reports?

- The Streamlit application reads the database through a **read-only reporting account** and keeps results in memory for fast filtering.
- Users open a **web link** in a browser and use the sidebar filters (period, source, destination, via, business, consignor, consignee, LR, target days). Every table can be downloaded as CSV.
- Live pages refresh on demand or on a schedule from SRD's APIs.
- Access can be restricted by company login (SSO) or email invitation. The exact method is a decision for SRD (see requirements).
- Because the data sits in standard PostgreSQL, SRD can also point other tools such as Power BI at it later.

What we need from SRD Logistics

Each item below is needed to move from the demonstration to production. Items marked **Before build** block the start of the build; the rest can follow.

#	Item	Why we need it	Needed
A. Data and samples			
A1	Sample Excel / CSV extracts of LR data: at least 3 months, ideally 12 months, with SRD's real column headings	To map SRD's fields, tune validation and replace the synthetic data	Before build
A2	Confirmation of which fields SRD can supply (see appendix), especially the timestamp at each journey stage	Stage-level delay analysis depends on stage timestamps	Before build
A3	Master lists: source branches, destinations, main hubs, via hubs, and the hub assigned to each route	Route, via and hub reports	Before build
A4	Consignor and consignee lists (with IDs, if available) and business / segment names	Customer reports and drill-down	Before build
A5	Delay reason list, IF-condition remarks and operational remarks, and the stage at which each is recorded	Reason % and remarks in every delay report	Before build
A6	Volumes: LRs per day, history to load, expected growth	Sizing of the database and hosting	Before build
B. Targets and business rules			
B1	Benchmark target days per route (source to destination) and target service % per route	Foundation of every target-vs-actual report	Before build
B2	Definition of service % and a delayed LR: calendar or working days, Sunday and holiday treatment, clock start (booking or collection), how pending LRs count	So our numbers match SRD's own reports	Before build
B3	Hub time limits: booking to main-hub receipt, receipt to dispatch; ageing buckets wanted	Booking to main hub report	During build
B4	Meaning of 'KT weight', 'via', and 'lagging / leading performance'	Avoids building the wrong measure	Before build
B5	Target-day views wanted (for example 4, 5, 6, 7 and above) and what makes a delay 'recurring' for suggestions	Filters and suggestion rules	During build
C. Live data (SRD application API)			
C1	API documentation: endpoints, fields, formats, and a test / sandbox environment	Live reports are built against the real API	Before build
C2	Authentication method and credentials (shared securely), rate limits, expected refresh frequency	Connection design and scheduling	During build
C3	Network requirements: IP whitelisting, VPN, firewall rules	So the dashboard host can reach the API	During build
D. Infrastructure and access			
D1	Hosting choice: DataQ-managed, SRD cloud, or on-premise; any security or data-residency policy	Real LR data includes customer names; hosting must be approved	Before build
D2	A PostgreSQL instance, or approval for us to provision one; backup expectations	Permanent storage	During build
D3	User list and roles, login preference (SSO or email invitation), expected concurrent users	Access control and sizing	During build
D4	Company name, logo and preferred URL; GitHub organisation if SRD wants to own the code	Branding and ownership	During build
E. People and process			
E1	A named business contact (operations) and a technical contact (IT / data)	Fast answers on definitions and access	Before build
E2	Who will upload data, how often, and the daily cut-off time	Design of the upload workflow and refresh schedule	During build

#	Item	Why we need it	Needed
E3	UAT participants, turnaround time for feedback, and sign-off criteria	Predictable acceptance	UAT
E4	Export formats needed beyond CSV (Excel, PDF)	Scope of reporting outputs	During build

Appendix: LR data fields

The dashboard works from one row per LR. **Required** fields are needed for the basic service reports; **Recommended** fields unlock stage-level delay, hub and via reports; **Optional** fields enrich the analysis. SRD's own column names will be mapped to these in the Upload page.

Field	Meaning	Priority
LR_No	Unique LR (lorry receipt) number	Required
Booking_DT	Date and time the LR was booked	Required
Delivery_DT	Date and time of delivery (blank until delivered)	Required
Source / Destination	Booking branch and delivery city	Required
Business	Business or segment (for example express cargo, part load)	Required
Consignor / Consignee	Sender and receiver names or IDs	Required
Collection_DT	Goods collected from the consignor	Recommended
Dispatch_DT	Dispatched from the source godown	Recommended
Hub_Receipt_DT / Hub_Dispatch_DT	Received at and dispatched from the main hub	Recommended
Via_Arrival_DT / Via_Dispatch_DT	Arrival at and departure from the via hub (if any)	Recommended
Dest_Arrival_DT / Dest_Received_DT	Vehicle arrival at destination, and inward entry at the destination godown	Recommended
Main_Hub / Via	Main hub and via hub on the route ('Direct' if none)	Recommended
Status	Current stage of the LR (booked, in transit, at hub, delivered, and so on)	Recommended
Delay_Reason	Reason recorded for the delay	Recommended
Condition_Remark	IF / condition-based remark for the delay	Recommended
Operational_Remark	Action taken or operational note	Recommended
Weight_KG / Articles	Weight and number of articles	Optional
Freight_Amount_INR	Freight charged	Optional
Distance_KM	Route distance	Optional

Targets (benchmark days and target service % per route) are supplied separately as a simple table: Source, Destination, Target days, Target service %.